Novosibirsk State University
*THE REAL SCIENCE

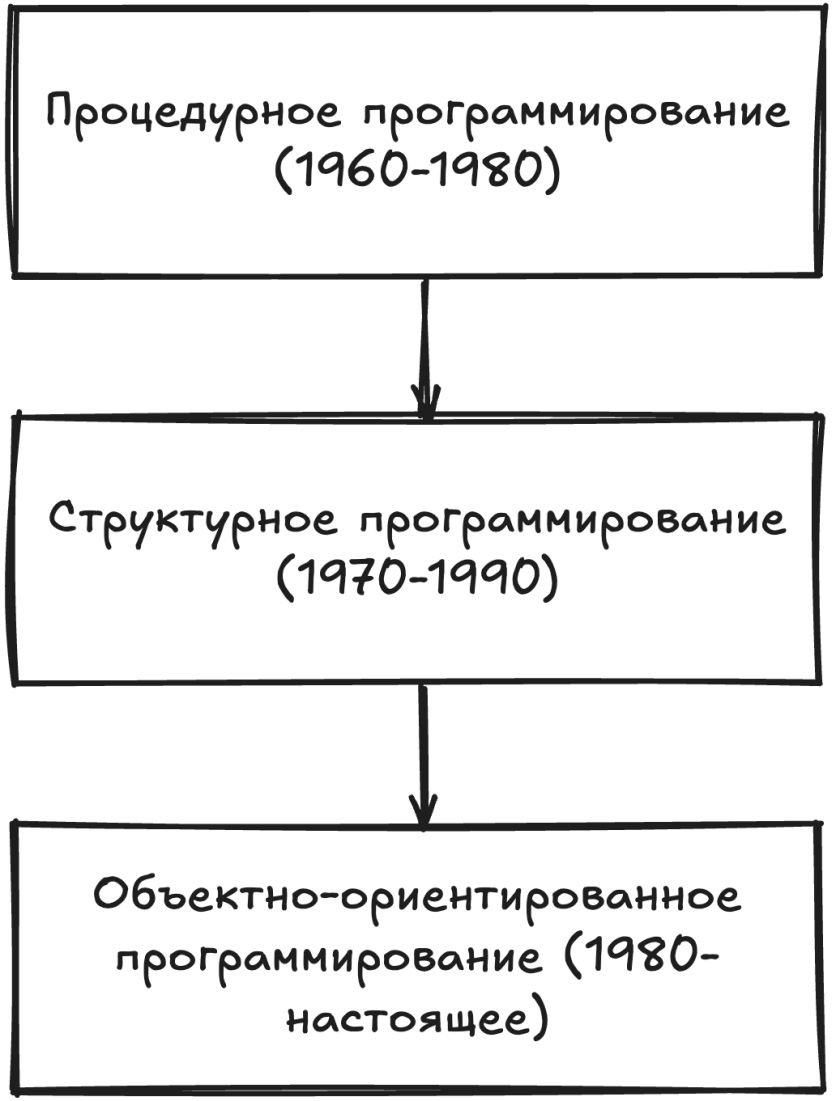
Программирование на Python

Объектно-ориентированное программирование на Python

План лекции

1. Введение в парадигмы программирования
2. Основные концепции ООП
3. Классы и объекты в Python
4. Атрибуты и методы
5. Наследование и иерархия классов
6. Полиморфизм
7. Инкапсуляция
8. Практические примеры
9. Лучшие практики

История: От процедурного программирования к ООП



Парадигмы программирования

Парадигма	Фокус	Пример
Процедурная	Функции и шаги	Pascal, C
Функциональная	Функции как данные	Lisp, Haskell
Объектно-ориентированная	Объекты и их взаимодействие	Java, C++, Python
Декларативная	Описание "что", а не "как"	SQL, HTML

Python поддерживает все эти парадигмы!

Ключевой вопрос: Зачем нужно ООП?

- **Организация**: Группировка связанных данных и операций
- **Масштабируемость**: Управление сложностью растущих проектов
- **Переиспользование**: Наследование и композиция
- **Поддержка**: Чистый, понятный код
- **Моделирование реальности**: Объекты отражают реальные сущности

Что такое объект?

Объект — это сущность, которая:

- Имеет **состояние** (атрибуты, данные)
- Имеет **поведение** (методы, действия)
- Имеет **идентичность** (уникальный адрес в памяти)

```
Пример: Автомобиль
├─ Состояние: цвет, марка, год, скорость
├─ Поведение: ехать, тормозить, поворачивать
└─ Идентичность: это конкретный автомобиль
```

Что такое класс?

Содержимое слайда: Класс — это:

- Чертеж, шаблон, архитектура для создания объектов
- Определение структуры (атрибуты) и поведения (методы)
- Абстрактное определение
- Невидим до создания экземпляра

Аналогия: Класс как архитектурный план дома, объект как конкретный построенный дом

Синтаксис Python — определение класса

- `class` — ключевое слово для определения класса
- `__init__` — специальный метод инициализации
- `self` — ссылка на текущий объект

```python
class Car:
    """Класс для описания автомобиля"""

    def __init__(self, brand, color, year):
        self.brand = brand      # атрибут
        self.color = color      # атрибут
        self.year = year        # атрибут
        self.speed = 0          # атрибут с начальным значением
```

Создание объекта (инстанцирование)

- Каждый объект имеет свои собственные атрибуты
- Они независимы друг от друга
- `()` вызывают метод `__init__`

```python
# Создание объектов класса Car
car1 = Car("BMW", "черный", 2020)
car2 = Car("Toyota", "серебристый", 2022)
car3 = Car("Lada", "синий", 2019)

# Доступ к атрибутам
print(car1.brand)    # BMW
print(car2.color)    # серебристый
print(car3.year)     # 2019
```

Методы класса

```python
class Car:
    def __init__(self, brand, color, year):
        self.brand = brand
        self.color = color
        self.year = year
        self.speed = 0

    def accelerate(self, amount):       # метод
        self.speed += amount
        print(f"Скорость увеличена на {amount}")

    def brake(self):                    # метод
        self.speed = 0
        print("Автомобиль остановлен")

    def get_info(self):                 # метод
        return f"{self.brand} {self.year} года"

# Использование методов
car1.accelerate(50)
car1.brake()
car1.get_info()
```

Четыре столпа ООП – Введение

1. ИНКАПСУЛЯЦИЯ (Encapsulation)
2. НАСЛЕДОВАНИЕ (Inheritance)
3. ПОЛИМОРФИЗМ (Polymorphism)
4. АБСТРАКЦИЯ (Abstraction)

Инкапсуляция (Encapsulation)

- public – `name`
- protected – `_name`
- private – `__name`

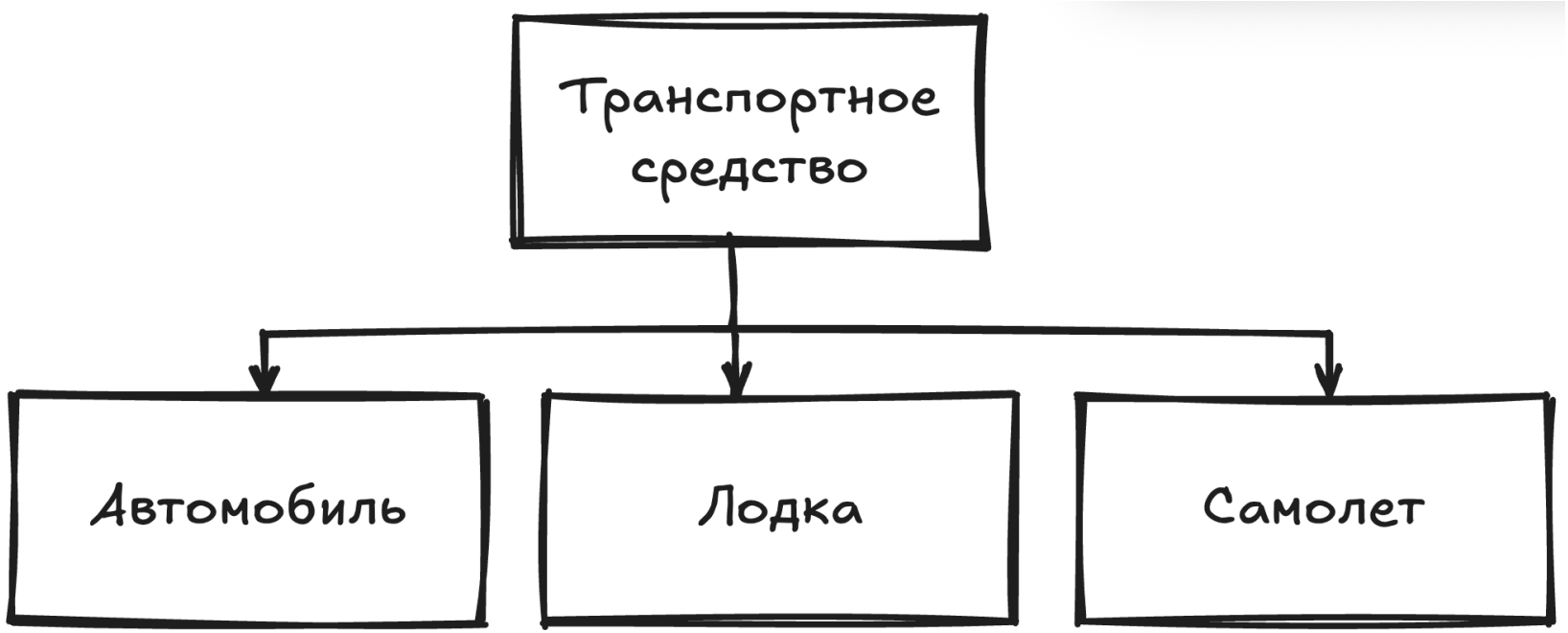
```python
class BankAccount:
    def __init__(self, owner, balance):
        self._owner = owner                 # защищенный атрибут
        self.__balance = balance            # приватный атрибут

    def deposit(self, amount):              # публичный метод
        if amount > 0:
            self.__balance += amount

    def get_balance(self):                  # публичный метод
        return self.__balance
```

Наследование (Inheritance)

- **Наследование** – это механизм создания новых классов на основе существующих.
- **Базовый класс (суперкласс)** – родитель
- **Производный класс (подкласс)** – потомок
- Подкласс наследует все атрибуты и методы базового класса



Наследование в Python

- `super()` – обращение к методам родительского класса

```python
class Vehicle:                          # базовый класс
    def __init__(self, brand):
        self.brand = brand

    def drive(self):
        print(f"{self.brand} едет")

class Car(Vehicle):                     # производный класс
    def __init__(self, brand, doors):
        super().__init__(brand)         # вызов конструктора родителя
        self.doors = doors

    def honk(self):                     # новый метод
        print("Бип-бип!")

car = Car("BMW", 4)
car.drive()                             # наследованный метод
car.honk()                              # собственный метод
```

Цепочка наследования (многоуровневое наследование)

- Можно строить цепочки наследования на несколько уровней

```python
class Vehicle:
    def __init__(self, brand):
        self.brand = brand

class Car(Vehicle):
    def __init__(self, brand, doors):
        super().__init__(brand)
        self.doors = doors

class SportsCar(Car):                    # наследует от Car
    def __init__(self, brand, doors, topspeed):
        super().__init__(brand, doors)
        self.topspeed = topspeed

sports_car = SportsCar("Ferrari", 2, 350)
```

Полиморфизм (Polymorphism)

- **Полиморфизм** — "многоформность", способность объектов разных классов реагировать на одинаковые команды по-разному.
- Один интерфейс, разное поведение
- Очень полезно для создания гибкого кода

```python
class Dog:
    def speak(self):
        return "Гав!"

class Cat:
    def speak(self):
        return "Мяу!"

def make_speak(animal):
    print(animal.speak())

make_speak(Dog())  # Гав!
make_speak(Cat())  # Мяу!
```

Переопределение методов (Method Overriding)

- Подкласс может переопределить метод родителя
- Если метода нет в подклассе, используется метод родителя

```python
class Animal:
    def speak(self):
        return "Издает звук"

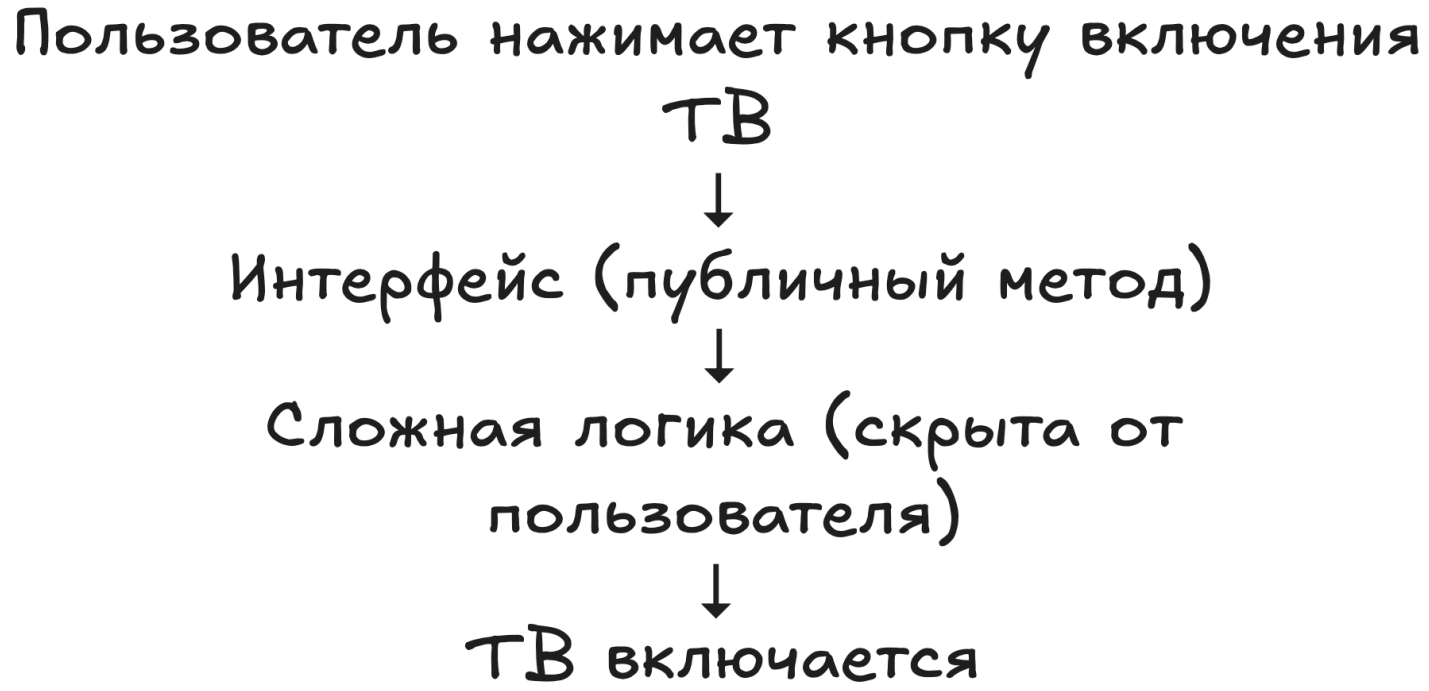
class Dog(Animal):
    def speak(self):                    # переопределение
        return "Гав! Гав!"

class Cat(Animal):
    def speak(self):                    # переопределение
        return "Мяу, мяу"

dog = Dog()
print(dog.speak())                      # Гав! Гав!
```

Абстракция (Abstraction)

- **Абстракция** – скрытие сложности, предоставление только необходимого интерфейса.
- Абстрактные классы и интерфейсы в Python
- `abc.ABC` и `@abstractmethod`



Абстрактные классы в Python

```python
import math
from abc import ABC, abstractmethod

class Shape(ABC):                          # абстрактный класс
    @abstractmethod
    def area(self):                        # абстрактный метод
        pass

    @abstractmethod
    def perimeter(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return math.pi * self.radius ** 2

    def perimeter(self):
        return 2 * math.pi * self.radius

# Shape() — ошибка! Абстрактный класс нельзя инстанцировать
circle = Circle(5)
print(circle.area())
```

Композиция vs Наследование

- НАСЛЕДОВАНИЕ: "является"
  - Car "является" Vehicle
- КОМПОЗИЦИЯ: "имеет"
  - Car "имеет" Engine, Wheels, Seats
- Композиция часто предпочтительнее наследования
- "Composition over inheritance"

```python
class Engine:
    def start(self):
        print("Двигатель запущен")

class Car:                              # композиция
    def __init__(self):
        self.engine = Engine()          # Car "имеет" Engine

    def start(self):
        self.engine.start()

car = Car()
car.start()
```

Магические методы (Dunder Methods)

- `__init__` – инициализация
- `__str__` – красивый вывод
- `__repr__` – представление для отладки
- `__len__` – длина
- `__eq__`, `__lt__` и т.д. – сравнение

```python
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):                  # строковое представление
        return f"{self.name}, {self.age} лет"

    def __repr__(self):                 # для отладки
        return f"Person('{self.name}', {self.age})"

    def __len__(self):
        return self.age

    def __eq__(self, other):            # сравнение
        return self.age == other.age

person = Person("Alice", 30)
print(person)                           # Alice, 30 лет
print(len(person))                      # 30
```

Свойства (Properties)

- Свойства — это управляемые атрибуты
- Позволяют добавить логику при чтении/записи

```python
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius

    @property                           # геттер
    def celsius(self):
        return self._celsius

    @celsius.setter                     # сеттер
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("Невозможная температура")
        self._celsius = value

    @property
    def fahrenheit(self):
        return self._celsius * 9/5 + 32

temp = Temperature(25)
print(temp.celsius)                     # 25
temp.celsius = 30                       # использует сеттер
print(temp.fahrenheit)                  # 86.0
```

Датаклассы (@dataclass)

- прощает создание классов для хранения данных
- Автоматически генерирует `__init__`, `__repr__`, `__eq__`, `__hash__`
- Требует Python 3.7+

```python
from dataclasses import dataclass

@dataclass
class Person:
    """Класс для хранения данных о человеке."""
    name: str
    age: int
    is_student: bool = False

# Не нужно писать __init__, __repr__, __eq__!
p1 = Person("Alice", 30)
p2 = Person("Bob", 25, is_student=True)
p3 = Person("Alice", 30)

print(p1)         # Person(name='Alice', age=30, is_student=False)
print(p1 == p3)   # True
```

Статические методы и методы класса

- `@staticmethod` — функция, не привязанная к объекту
- `@classmethod` — работает с классом, а не объектом
- Полезны для утилит и альтернативных конструкторов

```python
class MathUtils:
    pi = 3.14159

    @staticmethod
    def add(a, b):                      # не нужен self или cls
        return a + b

    @classmethod
    def from_radius(cls, radius):       # получает класс, не объект
        return cls(2 * 3.14159 * radius)

    def __init__(self, value):
        self.value = value

result = MathUtils.add(5, 3)            # 8
circle = MathUtils.from_radius(10)      # используя classmethod
```

Множественное наследование

- Класс может наследовать от нескольких родителей
- Порядок важен! (MRO — Method Resolution Order)
- Может привести к "ромбовидной проблеме"

```python
class Swimmer:
    def swim(self):
        return "Плывет"

class Flyer:
    def fly(self):
        return "Летит"

class Duck(Swimmer, Flyer):            # наследует от двух классов
    pass

duck = Duck()
print(duck.swim())                     # Плывет
print(duck.fly())                      # Летит
```

Примеси (Mixins)

- Mixins — это классы, которые добавляют функциональность
- Используются для переиспользования поведения
- Более гибкие, чем множественное наследование

```python
class TimestampMixin:                    # примесь (mixin)
    def get_timestamp(self):
        from datetime import datetime
        return datetime.now()

class LogMixin:                          # примесь
    def log(self, message):
        print(f"[LOG] {message}")

class User(TimestampMixin, LogMixin):
    def __init__(self, name):
        self.name = name

    def save(self):
        self.log(f"Пользователь {self.name} сохранен")
        self.timestamp = self.get_timestamp()

user = User("Alice")
user.save()
```

Документирование кода (Docstrings)

```python
class Calculator:
    """Простой калькулятор для базовых операций.

    Attributes:
        history (list): История операций
    """

    def __init__(self):
        """Инициализирует новый калькулятор."""
        self.history = []

    def add(self, a, b):
        """Складывает два числа.

        Args:
            a (float): Первое число
            b (float): Второе число

        Returns:
            float: Сумма чисел

        Examples:
            >>> calc = Calculator()
            >>> calc.add(2, 3)
            5
        """
        result = a + b
        self.history.append(f"{a} + {b} = {result}")
        return result

help(Calculator.add)
```

Лучшие практики ООП

1. **SOLID принципы:**
   - S: Single Responsibility Principle
   - O: Open/Closed Principle
   - L: Liskov Substitution Principle
   - I: Interface Segregation Principle
   - D: Dependency Inversion Principle
2. **Другие принципы:**
   - DRY (Don't Repeat Yourself)
   - KISS (Keep It Simple, Stupid)
   - Composition over Inheritance
   - Prefer Immutability where possible

Паттерны проектирования

- Паттерны проектирования - проверенные решения часто встречающихся проблем в ООП

```
Основные категории:
├─ Порождающие (Creational)
│  └─ Singleton, Factory, Builder
├─ Структурные (Structural)
│  └─ Adapter, Decorator, Proxy
└─ Поведенческие (Behavioral)
   └─ Observer, Strategy, Template Method
```

Паттерн Singleton

```python
class Singleton:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

singleton1 = Singleton()
singleton2 = Singleton()
print(singleton1 is singleton2)         # True

# Или через декоратор:
def singleton(cls):
    instances = {}
    def get_instance(*args, **kwargs):
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]
    return get_instance

@singleton
class Database:
    def __init__(self):
        self.connection = None
```

Паттерн Factory

```python
class Dog:
    def speak(self):
        return "Гав!"

class Cat:
    def speak(self):
        return "Мяу!"

class AnimalFactory:
    @staticmethod
    def create_animal(animal_type):
        if animal_type == "dog":
            return Dog()
        elif animal_type == "cat":
            return Cat()
        else:
            raise ValueError("Неизвестный тип животного")

# Использование
animal = AnimalFactory.create_animal("dog")
print(animal.speak())
```

Паттерн Observer

```python
class Subject:
    def __init__(self):
        self._observers = []

    def attach(self, observer):
        self._observers.append(observer)

    def notify(self):
        for observer in self._observers:
            observer.update(self)

class WeatherStation(Subject):
    def __init__(self):
        super().__init__()
        self._temperature = 0

    @property
    def temperature(self):
        return self._temperature

    @temperature.setter
    def temperature(self, temp):
        self._temperature = temp
        self.notify()

class TemperatureDisplay:
    def update(self, subject):
        print(f"Новая температура: {subject.temperature}°C")

station = WeatherStation()
display = TemperatureDisplay()
station.attach(display)
station.temperature = 25
```

Тестирование кода ООП

```python
import unittest

class Calculator:
    def add(self, a, b):
        return a + b

    def divide(self, a, b):
        if b == 0:
            raise ValueError("Деление на ноль")
        return a / b

class TestCalculator(unittest.TestCase):
    def setUp(self):
        self.calc = Calculator()

    def test_add(self):
        self.assertEqual(self.calc.add(2, 3), 5)

    def test_divide_by_zero(self):
        with self.assertRaises(ValueError):
            self.calc.divide(10, 0)

if __name__ == '__main__':
    unittest.main()
```

Заключение: Ключевые моменты

1. **ООП — это парадигма**, которая помогает управлять сложностью
2. **Четыре столпа:** Инкапсуляция, Наследование, Полиморфизм, Абстракция
3. **Классы и объекты:** Классы — это шаблоны, объекты — это экземпляры
4. **Композиция часто лучше наследования**
5. **SOLID принципы** — руководство к хорошему дизайну
6. **Паттерны проектирования** — проверенные решения
7. **Тестирование** — важная часть разработки